SC: grupo 1/Aula 1

decimal- $(_2$ - $(_{16}$ = todos son sist. posicionales

Computadora → ejecutor programas. Almacenados en memoria en binario → Es lo + facil de identificar

4 tipos de datos → N° enteros sin/con signo - Reales con signo - Decimales codificados en binario (BCD) - caracteres

→ Sin signo, módulo y signo, Ca 1 (complemento a la base reducida) - Ca 2 (complemento a la base) - exceso

3/7 → Tarea práctica 5

14/7 → Evaluax tp.

5/8 → Evaluax teoría (por promax)

4/8 → Recuperatorio tp.

18/8 → Recuperatorio tp.

Binarios → N° enteros sin signo: de 0 a + Rango de 0 a $2^n-1$

Ejemplo n=8 bits
0 00000000
128 10000000
255 11111111

Teorema Fundamental de la Numerax $N° = \sum_{i=m}^{n} (\text{dígito})_i \times (\text{base})^i$

$$x_4 \times B^4 + x_3 \times B^3 + x_2 \times B^2 + x_1 \times B^1 + x_0 \times B^0 + x_{-1} B^{-1} + \dots$$

Ejemplo: base 10

$$\left.\begin{array}{l} 3574: 3000 + 500 + 70 + 4 \\ 3 \times 10^3 + 5 \times 10^2 + 7 \times 10^1 + 4 \times 10^0 \\ \text{3 unidades de mil + 5 centenas + 7 decenas + 4 u.} \end{array}\right\}$$

$$\left.\begin{array}{l} 3.1416 = 3 \times 10^0 + 1 \times 10^{-1} + 4 \times 10^{-2} + 1 \times 10^{-3} + 6 \times 10^{-4} \\ \text{3 u. + 1 décimo + 4 centésimos + 1 milésima + 4 diezmilésimas} \end{array}\right\}$$

Números en punto fijo: se considera q' todos los números a representar tienen exacto' la misma cant. de dígitos y la coma está ubicada en el mismo lugar

Ej. balanza

Rango: de donde hasta donde puede abarcar c/los dígitos q' cuenta

Resolux: dif. e/ 2 n° consecutivos.

Ej: 0,00 a 9,99. Resolux: 0,01 9,99 - 9,98 = 0,01 }

NOTA

Operaciones ~~[illegible]~~ aritméticas → Realizada x la ALU

- Suma en binario: hay acarreos e/ posiciones Ej 1+1=10 (1 y me llevo 1)
- Resta: 0-0=0 1-0=1 1-1=0 0-1= el de la izquierda me presta 2.

Bits de condix (Banderas) Bits q' el procesador establece acorde al rtdo. de cada op. realizada.

- Permite tomar decisiones como: hacer o no una transferencia de control, determinar rtdo. el nº. Saltos condicionales nos lleva a una parte del codigo s/rtdo.

Z: vale 1 si el rtado de la operox son todos 0

C (carry): en la suma vale 1 si hay acarreo en el bit + significativo.
en la resta vale 1 si hay "borrow" hacia el bit + significativo
minuendo < al sustraendo

Ej.
$$\begin{array}{r} 1001 \\ +\,1010 \\ \hline \textcircled{1}0011 \end{array} \quad \text{4 bits.} \qquad \begin{array}{r} 1001 \\ -\,1100 \\ \hline \textcircled{1}1101 \end{array}$$
Flag acarreo — Flag carry

Sistema hexadecimal$_{16}$: Base 16

Digitos: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F}

Ej: $2CA,8_{10} = 2\times16^2 + C\times16^1 + A\times16^0 + 8\times16^{-1}$
$= 512 + 192 + 10 + 0,5$
$= 714,5_{10}$

$(2 <=> 16 = 2^4 = 16 \rightarrow$ 4 bits.

Si me falta, agrego 0 en los grupos de 4

| 10 | 2 | 10 |
|---|---|---|
| 0 | 0000 | 0 |
| 1 | 0001 | 1 |
| 2 | 0010 | 2 |
| 3 | 0011 | 3 |
| 4 | 0100 | 4 |
| 5 | 0101 | 5 |
| 6 | 0110 | 6 |
| 7 | 0111 | 7 |
| 8 | 1000 | 8 |
| 9 | 1001 | 9 |
| A | 1010 | 10<br>11 |
| B | 1011 | 12 |
| C | 1100 | 13 |
| D | 1101 | 14 |
| E | 1110 | 15 |
| F | 1111 | 16 |

p.45.

$451,75_{10}$ $\quad$ $\begin{array}{r} 451 \\ -256 \\ \hline =195 \end{array}$ $\quad$ 111000011,11

BCD: ┬ E/S periféricos: los nº se codifican usando un byte por digito. Se dice q' el nº está desempaquetado.
↓ └ En calculo, se reservan 4 bits x digito. Se dice q el nº esta empaquetado (doble q' la vez)

Decimal codificado en binario

Sin signo: p/ emisor y receptor c/ <> reloj.

Los 4 1º 1 diferencian al digito

Ejemplo: 834: 11111000 11110011 11110100
= F8 F3 F4

Con signo: +834 = 11111000 11110011 11000100
F8 F3 C4

Los 4 bits q' acompañan al ultimo digito son reemplazados x el signo

-834 = 11111000 11110011 11010100
F8 F3 D4

C = signo + 1101
D = signo -

Empaquetado c/ signo:
+834 = 10000011 01001100 83 4C
-34 = 00000011 01001101 03 4D

Suma en BCD

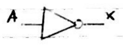
| | | |
|---|---|---|
| 41 | 0100 0001 | |
| +22 | 0010 0010 | |
| 63 | 0110 0011 | |
| +15 | 0001 0101 | |
| 27 | 0010 0111 | |
| 42 | 0011 1100 | > 9 |
| | 0110 | → +6 |
| | 0100 0010 | |

NOTA

# Nivel de lógica digital

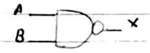
- Un circuito digital es en el q' estan presentes dos valores lógicos.
- Compuertos son dispositivos electrónicos q' pueden realizar <> funciones c/ estos 2 valores lógicos
- Compuertos básicas: AND, OR, NOT, NAND, NOR, XOR

Compuertas simbolo y descripx funcional.



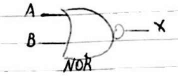
NOT

| A | X |
|---|---|
| 0 | 1 |
| 1 | 0 |



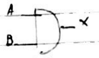
NAND

| A | B | X |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |



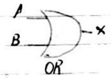
NOR

| A | B | X |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |



AND

| A | B | X |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |



OR

| A | B | X |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

XOR

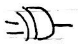
2 entradas = rtdo 0

" <> rtdo. 1

XNOR

= rtdo 1

<> rtdo. 0



Algebra de Boole: leyes matemáticas q' gobiernan el comportamiento de las operaciones c/ bits y describen circuitos

De Morgan $\overline{A.B}=\overline{A}+\overline{B}$ / $\overline{A+B}=\overline{A}.\overline{B}$

- $\overline{\overline{A}}=A$

Identidad: $1.A=A$ / $0+A=A$

Nula $0.A=0$ / $1+A=1$

Idempotencia $A.A=A$ / $A+A=A$

Inversa $A.\overline{A}=0$ / $A+\overline{A}=1$

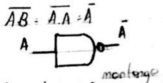
Conmutativa $A.B=B.A$ / $A+B=B+A$

Asociativa $(AB).C=A(BC)$ / $(A+B)+C=A+(B+C)$

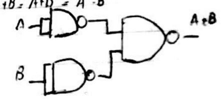
Distributiva $A+B.C=(A+B).(A+C)$ / $A.(B+C)=AB+AC$

Absorcion $A.(A+B)=A$ / $A+A.B=A$.

Ejemplo: • construir un NOT c/ NAND $F=\overline{AB}=\overset{\text{idempotencia}}{\overline{A.A}}=\overline{A}$



• Construir OR con NAND $F=A+B=\overline{\overline{A+B}}=\overline{\overline{A}.\overline{B}}$ mantengo equivalencia y uso ley de Morg



- Escribir la tabla de verdad p/ la fx
- Dibujar una AND p/ cada término que tiene 1 en la columna de rtdo.
- Invertir E necesario
- Unir todas las AND a una OR

Recordatorias

- En una AND, basta q una de sus variables de E sea 0 p/ q la fx = 0
- En una OR
-
-

Practica ①

1- Representar en el sist. BSS a 8 bits

1 = 0000 0001 35 = 0010 0011 No puedo representar -255, 256, -1, -8,

127 = 0111 1111 100 = 0110 0100 -127, -128, -199, -256, -100, 0,5, 1,25

128 = 1000 0000 xq exceden los 8 bits o es negativo, o [illegible]

137 = 1000 1001 o de [illegible] BSS No Permite

2- Interpretar cadenas de 8 bits en BSS

0000 0000 = 0 1000 0000 = 128 1111 1111 = 255 0111 1111 = 127

[illegible] 0011 = 35 1111 1110 = 254 1010 1010 = 170 [illegible] 0110 = 102

3- 111111, 1111 = 63, 9375 Rango: Resolux = 0,0625

5- Interpretar con el pto. 3

00 0000,0000 = 0,0 [illegible] 32,0 [illegible] 31,375

[illegible] 27,375

21,3125 63,875 [illegible]

6- 0

BCS, Co1, Co2 = nº +

. Tecnica de complemento: . Complemento de N de a = N-A

. Complemento a N de (N-A) = N - (N-A) = A

Clase ② → Números c/ signo. →

- Representar en BCS. o Modulo y signo.
  - Con n bits, 1 bit representa al signo y n-1 bits, la magnitud
  - El bit extremo izquierdo representa solo al signo.
  - Los bit 0 a n-2 la magnitud

$\underset{\text{signo}}{\overset{n-1}{}}\ |\ \overset{n-2}{\text{magnitud}}\ \ 0$

Ej: 0010 = +2
1010 = -2

- 0: positivo 1 = negativo
- Rango $-(2^{n-1}-1) \rightarrow +(2^{n-1}-2)$

Doble representax del 0 → 1000 y 0000

- Complemento a 1 de $m = 2^{n-1} - m$
- Complemento a 2 de $m = 2^{n} - m$

- Representax en Co1: 
  - 0: + 1: -
  - No se puede desglosar como en BCS $\overset{n-1}{}$ número 0
  - El Co1 de un nº en base 2 se obtiene invirtiendo todos los bits
  - Rango $-(2^{n-1}-1)$ a $+(2^{n-1}-1)$

Ej: +32 = 0010 0000
-32 = 1101 1111
+7 = 0000 0111
-7 = 1111 1000
n = 8 bits

$N^{\circ} - \begin{cases} 1111\ 1111 = -0 \\ 1000\ 0000\ \ -(2^{n-1}-1) = -127 \end{cases}$

$N^{\circ} + \begin{cases} 0111\ 1111\ \ +(2^{n-1}-1)\ +127 \\ 0000\ 0000 = +0 \end{cases}$

- Dada una cadena de bits ¿Qué nº decimal representa si lo interpretamos en Co1?

$+ = 0110\ 0000 = 1 \times 2^{6} + 1 \times 2^{5} = 64 + 32 = 96$

-, Puedo hacer 2 casos:

1º Co1 del nº y obtengo el + = 1110 0000 -31
0001 1111 +31

2° El peso que tiene el 1° dígito es $-(2^{n-1}-1)$ y el resto +.
$1110\ 0000 = -1\times(2^7-1)+1\times2^6+1\times2^5 = -127+64+32=-31$

· **Representox en Co2:** · Se escribe el n° = hasta encontrar el 1° 1, y luego se invierte
Si rtdo -, es el Co2 -
· Defnix del complemento
$Co2 = b^n - N°$
· La suma se efectúa = q' si los n° fuesen ℤ sin signo. Si el rtdo es +, el n° es el Co2 +, q' es el ℤ sin signo
Co2 = Co1 + 1
· Rango: $-(2^{n-1})$ a $+(2^{n-1}-1)$ → Asimétrico pq' tiene un solo 0.
· Hay un solo 0.
Co1 + 1.
· Negativo a Co2: ~~Co1~~ ~~[illegible]~~ y súmale 1

n = 5
$-10 = 2^5 - N°$
$100000 - 01010 = $ Co2
10110

Ej: +32 = 0010 0000
1101 1111 → Co1
1
· Al sumar 2 n° en Co2 tmb. incluye en la opera... 1 → sumo 1 el bit del signo. El
1110 0000 acarreo producido x el bit de signo se desprecia

· Regla de desbordamiento: si se suman 2 n° c/ el mismo signo, solo se produce desbordamiento si el signo del rtdo es <>
· ¿Qué n° decimal represento si lo interpretamos en Co2?
1° $0110\ 0000 = 1\times2^6+1\times2^5 = 64+32=96$ → Si es +

2° Co2 el n° y obtengo el + $1110\ 0000 = -32$ → si es -
$0010\ 0000 = +32$
· Sustrox: Para restar B de A, efectuar el complemento a 2 de B y sumarlo a A.

3° El peso que tiene el 1° dígito ahora es $-(2^{n-1})$ y el resto de los dígitos +
$1110\ 0000 = -1(2^7)+1(2^6)+1.2^5 = -128+64+32=-32$
· Resta: se obtiene el Co2 del sustraendo y se le suma el minuendo
· **Técnica de exceso:** · La representox de un n° A es la q' corresponde a la suma del mismo y un valor E
Exceso E de A = A+E
· Dado el valor, el n° representado se obtiene restando el valor del exceso
· Único representante del 0. A = (Exceso de E de A) - E
~~Ej: 10(10) en 8 bits: 128+10 = 138~~ = exceso de Co2
· Rango: $-(2^{n-1})$ a $+(2^{n-1}-1)$
n = 4
Ej: $28_{(10)} = 28-16 = 12$ Se corre la recta
· Si empiezo con 0 = - y con 1 = +
· Si un n° en exceso empieza c/ 0 es - y p/ saber su valor se calcula $(V/2 - 0)$

---

antes de un else no usar ; xq' corta
· Capac. de representox: n° q' se pueden representar

Banderas aritméticas: N: si el bit + significativo 0 = + 1 = N.
V = Overflow - condición fuera de rango · En Co2 representa q' el rtdo es erróneo

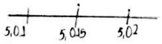
## Clase 3 → Representox de n° en Pto Flotante

Surgen de la necesidad de representar n° ℝ c/ un rango de representox > al q' nos permite el pto fijo

· Todos los n° a representar tienen la = cant. de dígitos y la coma fraccionaria está siempre en el mismo lugar
· La dif. e/ la representox en el papel y en la PC, es que no se guarda coma alguna, se supone q' está en un lugar dtdo.
Rango: -31,875 - 31,875
Rango: dif e/ n° > y <
Resolux: e/ 2 n° consecutivos → Viene dado por el n° - significativo

Error de pto fijo: El máx error cometido en una representox puede considerarse como la mitad de la diferencia (resolux) e/ 2 n° consecutivos

$\pm S . B^{\pm E}$ → Exponente
S ↓ Matisa
B ↓ Base implícita



5,01 ≤ n° < 5,015 se representa x 5,01
5,015 < n° ≤ 5,02 se representa < 5,02
· En cualquiera de los 2 casos el Error Absoluto máx resulta ser
EA máx = 5,015 - 5,01 = 0,005 o (5,02 - 5,01)/2 = 0,005

N° en pto flotante: · En pto fijo un rango de ℤ + y negativos centrados en 0.
Suponiendo un n° c/ componente fraccionario, en este formato de pto. fijo tmb. puede representar n°.
Limitaciones "n° muy grandes y muy pequeños"
→ Permite tener un rango de n° muy pequeños o grandes y ser representados c/ pocos dígitos
No es necesario almacenarlo.
· 976 000 000 000 000 = 9,76 · $10^{14}$ → Exponente
↓
Mantisa
$\pm M \times B^{\pm E}$

- M y E representados en BSS, BCS, Ca2, Ca1, Exceso
  exponente, mantisa

Supongamos el sig formato en pto flotante

Mantisa { BSS, 4 bits, entera    Exponente { BSS, 4 bits, entera

Máximo: $1111 \times 2^{1111}$    Rango: $15 \times 2^{15}$

Mínimo 0    Resolux extremo sup. $[0, \dots, 15 \times 2^{15}] : [0, \dots, 491520]$

" inf. $R: (1-0) \times 2^0 : 1$

$R: (15-14) \times 2^{15} : 1 \times 2^{15}$

- Mantisa y exponente en Ca2

Mantisa { Ca2, 4 bits, Entero    Exponente { Ca2, 4 bits, Entero

Maximo $= 0111 \times 2^{0111} = +7 \times 2^{+7}$

Mínimo $= 1000 \times 2^{0111} = -8 \times 2^{+7}$

Rango $= [-8 \times 2^{+7} \dots +7 \times 2^{+7}]$

Resolux en el extremo superior $R: 7-6 \times 2^7 = 1 \times 2^7$

" en el origen: $R: (1 \times 2^{-8} - 0) = 1 \times 2^{-8}$

- Mantisa fraccionaria todos los bits son fraccionarios

Mantisa { BCS, 23 bits, fraccionario, 1 bit signo    Exponente { Ca2, 8 bits, Z

* Resolux siempre *

- Máximo positivo $0,111 \dots 111 \times 2^{01111111} = +(1-2^{-23}) . 2^{127}$
- Mínimo + (≠0) $0 \; 0,000 \dots 001 \times 2^{10000000} = +(2^{-23}) . 2^{-128}$
- Máximo negativo (≠0) $1 \; 0,000 \dots 001 \times 2^{10000000} = -(2^{-23}) . 2^{-128}$
- Mínimo negativo $1 \; 0,111 \dots 111 \times 2^{01111111} \; -(2^{-23}) . 2^{127}$

Formato final    Signo $\overset{0\,1}{,}$ exponente $\overset{8\;\;9}{,}$ Mantisa $^{31}$

Mínimo negativo 1 ,01111 111 ,1111 1111 1111 1111 1111 111

Normalización: $40 . 10^0 = 4 \times 10^1 = 0,4 \times 10^2 = 400 \times 10^{-1}$

- Existen <> valores de mantisa y exponente p/ representar un mismo nº

$\pm 0,1bbb \dots b \times 2^{\pm b}$ • C/ el obj. de tener un único par de valores de mantisa y exponente p/ un número, se introduce la normalizax. → solo p/ fraccionarios

- C/ el obj. anterior, las mantisas fraccionarias se definen como $0,1dddddd$ d es un dígito binario q' vale 0 o 1

Ejemplo

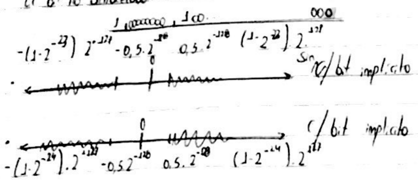
Mantisa { BCS, 23bits, Fraccionario, 1bit signo, Normalizada    Exponente { Exceso, 8bits, Entero

- Máx. + $0 \; 0,111 \dots 111 \times 2^{11111111} = +(1-2^{-23}) . 2^{+127}$
- Mín. + $0 \; 0,100 \dots 000 \times 2^{00000000} = +(0,5) . 2^{-128}$
- Máx. negativ. $1 \; 0,100000 \times 2^{00000000} \; -(0,5) . 2^{-128}$
- Mín. − $1 \; 0,1111 \dots 111 \times 2^{11111111} \; -(1-2^{-23}) . 2^{+127}$

Rango $[-(1-2^{-23}) . 2^{127}, -(0,5) . 2^{-128}] [+(0,5) . 2^{-128}, +(1-2^{-23}) . 2^{127}]$

Bit implícito: Si no lo almacenas puedo agregar un bit en la mantisa.
El bit no almacenado se lo conoce como bit implícito

Ejemplo:



Resolver [illegible] $(-0,5 \times 2^{-128})$ [illegible] $= 2^{-23}$ → dif. entre cada nº

y $2^{-23} \times 2^{-128}$ [illegible] $= 2^{-151}$ nº muy juntos

nº + grande $= (1-2^{-23}).2^{127}$ y $1,11\ldots = 2^{-23}$

$= 2^{104}$ $1,11\ldots$ → el mayor bit - significativa

diferencia e/ cada bit

* Dif e/ cada mantiza es el bit - significativo = bit - significativo . $2^{\text{exponente menor}}$

* Resolver extremo inferior = bit - s' . $2^{\text{exponente + grande}}$

¿Cómo se escribe un Nº en pto. flotante normalizado?

1- Se escribe el nº en el sist. p/ la mantisa
2- Se desplaza la coma y se cambia el exponente
3- Se convierte al exponente al sist. p/ el

-13,5

1- 1 1101,100...0 = 1 1101,100...0 × $2^0$
2- 1 0,110110...0 × $2^4$
3- 4 en Ca2 = 00000100

en exceso: 10000100

1 ,10000100 ,1101100... 00 sin Implícito

1 , 10000100 ,101100... 00 con Bit implícito

Error absoluto: EA máximo ≤ resolución/2
E relativo: EA / número a representar → da error en un nº

Muestra la precisión

Ej: balanza panadería / joyería

> valor < error

< valor > error ej. 0,5 e/ 1 y 2

1/2 = 0,5 = EA

Error relativo:

EA / 0,5 = 1

convención: ambos usuarios de acuerdo p/ una representar

Elige una norma

Estándar IEEE 754 Mantisa: fraccionaria normalizada, c/ coma dsp del primer bit
Exponente: representada en exceso $2^{n-1}-1$

| | Simple 32 bits | Doble presición 64 bits |
|---|---|---|
| Bits en signo | 1 | 1 |
| en exponente | 8 | 11 |
| En fracción | 23 | 52 |
| Total | 32 | 64 |
| Exponente en exceso | 127 | 1023 |
| Rango en exponente | -126 - +127 | -1023 +1023 |
| Rango de nº | $2^{-126}$ a $\sim 2^{128}$ | $2^{-1022}$ a $2^{1023}$ |

Combinac. de bits p/ casos especiales

1 , 8 , 23 Simple

1 , 11 , 32 doble

3F800000 = 0011 1111 1000 0000 0000 0000 0000

01111111 = 127 en exceso 127 = 0

1,0 × $2^0$ = 1

C0066666 = 1100 0000 0000 0110 0110 0110 0110 0110

10000000 = 128 en exceso 127 = 1

-1,05 × $2^1$ = -2,1

Casos especiales

1- Exponente: 255 o 2047. $M \neq 0$ => NaN "Not a number"
2- Exp. 255/2047. $M = 0$ => $\infty$
3- Exp: 0 $M \neq 0$ => Cero
4- Exp. 0 Mant $\neq 0$ => Denormalizado: n° + pequeño del q' queo representar
 • $\pm 0$, mantisa $= s \cdot p \; 2^{-126}$ c/ mantisa $0,$

Operaciones aritméticas en pto flotantes

1° debo igualar los exponentes → Hace corrimientos p/ exponente en común

Suma y resta: Cuando los exponente son =, saco factor común y sumo o resto M. El rdo. es $\pm M \cdot 2^{\text{exponente común}}$.

Multiplicar y división:
- Sumar y resta exponentes.
- Multiplicar y dividir mantisas s/ signo.
- Normalizar
- Redondear
- Todos los rdos intermedios deben [illegible] longitud p/ almacenarse.

$X = x_s \cdot B^{x_s}$ y
$y = y_s \cdot B^{y_s}$

$$\left.\begin{aligned} x+y &= (x_s \times B^{x-y} + y_s) \cdot B^{y_s} \\ x-y &= (x_s - B^{x-y} - y) \times B^{y_s} \end{aligned}\right\} x_E \leq y_E$$

$$x \cdot y = (x_s \times y_s) \times B^{x_s + y_s}$$

$$\frac{x}{y} = \left(\frac{x_s}{y_s}\right) \cdot B^{x_s - y_s}$$

---

• Conjunto de puertas interconectadas cuya salida es f° de la E de ese instante
↓ La aparic. de la E viene seguida [illegible] de la S
 c/ retardos de puertas

## Clase 4: circuitos lógicos combinacionales y secuenciales:

↓
- Responden a valores lógicos en E.
- Si cambia la E, cambia S
- Los val. anteriores de las E, no influyen los v. de S.

Se representa mediante tabla de verdad, ecuac booleana o circuito.

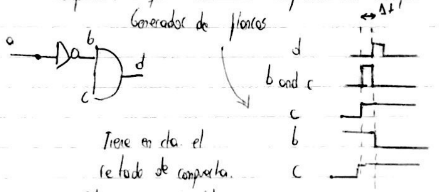
Cada [illegible] cambia la [illegible] instantáneamente [illegible] tiempo de propagación [illegible] a través de la puerta (retardo de puerta)

Ti a la dif. de la próxima hoja

• Circuito multiplexor de 8E/canales: tiene muchas E y una sola S.
 P/ seleccionar E, tiene 3 líneas de selección o selector de datos.
 ↓ Ej. • televisor, selecciona e/ varias E p/ 1 sola salida.
 • Internet: compartimos el ancho de banda simultáneo' usamos el mismo IR y se distribuye p/ los dispositivos.
 • Existe tmb. multiplexar en tiempo.

ALU hace operaciones.
Si se hace + y −, puedo hacer el resto de op.

Repuesta temporal: lo q' tardan en responder las compuertas.



• El retardo de compuertas es =

Tiene en cta. el retardo de compuerta.

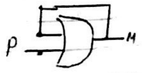
P/ crear una señal q' será un pulso de poca durac q' sincronizan o son un reloj p/ otros circuitos.

- PC: almacena y procesa
  ↳ Se necesitan circuitos capaces de registrar su estado anterior.

Así surgen los circuitos secuenciales

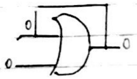
- Dependen de su E anterior y E interno del circuito
  ↳ Son las q' tienen la capacid de almacenar valores lógicos
- S depende de E

¿Como se almacena un valor lógico?



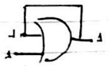
- La S es tmb. E
- En ningún circuito combinatorio una S transporta info, a una E
- La ecuac lógica $M = M+P$.

Ejemplo



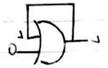
$P = 0$ y $M = 0$

$M = M+P = 0+0 = 0$



$P = 1$

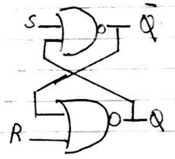
$M = M+P = 1+1 = 1$



$P = 0$

$M = M+P = 1+0 = 1$

FLIP-FLOP SR → tipo asincrónico ✓ son biestables → 2 S complementarios



| SR | $Q_{n+1}$ |
|---|---|
| 0 0 | $Q_n$ |
| 0 1 | 0 |
| 1 1 | 1 |
| 1 1 | Prohibido. |

Prohibido → pq $Q$ y $\bar{Q} = 1$

n: instante actual, para n+1 = próximo al actual

- S: Set: poner a 1
- R: Reset: " 0.
- $Q$ y $\bar{Q}$ complementarios

Los secuenciales se pueden clasificar en:

**Asincrónicas:** el cambio de la S, ocurre al cambio de la E + S anterior.

Sincrónicos: la S no cambia en el momento, sino cambian a partir de una señal de reloj.

Reloj: "señal especial"

- Temporiza eventos del sist.

T largo: frecuencia baja
T corto frecuencia alta.
} ciclo de tbjo

Ciclo de tbjo: q' % del periodo la onda está alta o baja.

FLIP FLOP D=
$\downarrow S \quad R = \bar{D}$

FLIP FLOP JK.
$\overset{\downarrow}{0} = \overset{\downarrow}{1} = 0.$

FLIP FLOP T Q cambia de 0 a 1 o 1 a 0 en cada pulso de la E de T

Recordando un bit: p/ recordar un bit en el registro tengo q' usar un Flip flop D

Selección y operaciones:
↳ identifica una celda.

Contador módulo 8 3 FLIP FLOP T. contador módulo 8.

[illegible]

Circuito combinacional
Entrada → C. combinacional → Salida

Circuito secuencial
Entrada → Circuito combinacional → Salida
Memorias

↗ Báscula q' senso 2 valores
Flip flop RS → set (fijar) → poner a 1
✓ Reset (Limpiar) → poner a 0

AND

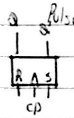
| | S | R | Q | Q' |
|---|---|---|---|---|
| Ajuste | 1 | 0 | 1 | 0 |
| Memoriza | 1 | 1 | Memoriza, vuelve al E q' estaba | |
| Limpia | 0 | 1 | 0 | 1 |
| Memoriza | 1 | 1 | 0 | 1 → E anterior antes de cp |
| | 0 | 0 | Indefinible | |

OR

| | S | R | Q | Q' |
|---|---|---|---|---|
| Set | 0 | 1 | 1 | 0 |
| Memoria | 0 | 0 | Val. | anterior |
| Resetear | 1 | 0 | | |
| Memoriza | 0 | 0 | | |
| Ajuste | 1 | 1 | Indefinible. | |

Flip flop con reloj ↳ Pulso del reloj.
Pulso bajo = 0.
Pulso alto = 1

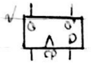
Q' funcione el flip-flop depende del reloj.



OR

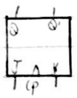
| Q | S | R | Q' |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | Indeterminado |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | |
| 1 | 1 | 1 | ind. |

FLIP FLOP D → Dato de entrada



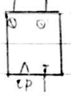
| Q | D | Q(t+1) |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

FLIP FLOP JK → Reset
↓
set



| | Q | J | K | Q(t+1) |
|---|---|---|---|---|
| Memoriza | 0 | 0 | 0 | 0 |
| Fijo | 0 | 0 | 1 | 0 |
| Limpia | 0 | 1 | 0 | 1 |
| Complementa | 0 | 1 | 1 | 1 |
| | 1 | 0 | 0 | 1 |
| | 1 | 0 | 1 | 0 |
| | 1 | 1 | 0 | 1 |
| | 1 | 1 | 1 | 0 |

FLIP FLOP T → Toggle lo contrario



| Q | T | Q(t+1) |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

0 → memoriza
1 → complementa

Clase 5

Informax automatica → [illegible]

Ciencia. tratamiento [ Racional / Automatico > Informax [illegible] Instrucciones

flexibilidad

Software – Hardware [illegible]

lógico equivalentes

Ciencia <— Analizar / Realizar —> Computadoras

Computadora es una maquina (sin inteligencia propia), digital (opera en el mundo digital). sincrónica (op controlada x señales). calculo numérico y lógico (puede modificar D de E), controlado por un programa (permite q la prog máquina opere de manera autonoma) q se almacena en una memoria. Y tiene comunicax c/el mundo exterior.

Arquitectura y organizax ¿[illegible]?

Arquitectura → atributos visibles al programador. Ej: instrucciones, nº bits usados p/ datos, mecanismos I/S, técnicas de direccionamiento.

¿Que me ofrece?

Organizax → como son implementados esos atributos Ej. señales de control, interfaz, tecno de memoria.

Ej. procesar suma 1+3 ? Como se [illegible] la suma [illegible]

[illegible]

- Toda la familia Intel x86 comparte la misma arquitectura basica
- flia. IBM System/370 comparte la arq. basica.

} 
- Brinda compatibilidad de código
- Surgen nuevos p/modificar limites del procesador anterior
- La org. difiere c/ versiones.

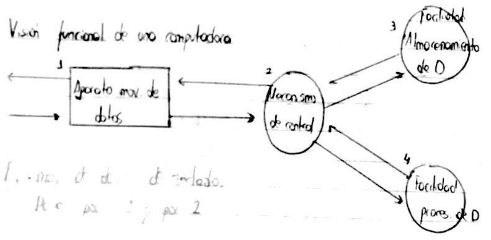
Ej. [illegible] puede tener [illegible] c/ nuevas versiones.

[illegible] 2017 [illegible] nueva flia.

Estructura y fx Estructura → modo en el cual se relacionan los componentes e/ sí. Funx → operax de los componentes individuales como parte de la estructura

- Procesamiento de D [illegible]
- Almacenamiento de D e Instrucciones en una memoria
- Movimiento de D [illegible] comp / S de D al usuario
- Control [illegible]

Visión funcional de una computadora

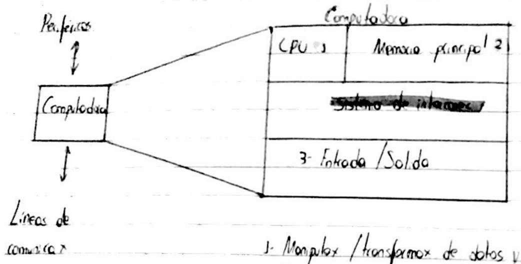


Ej. [illegible] de teclado. Pasa por 1 y por 2

- Almacenamiento, descarga de Internet a disco 1 → 2 → 3
- [illegible] hace almacenamiento de actualizar [illegible] bancarias 3 → 2 → 4
- Procesamiento desde almacenamiento a E/S (impresora) 3 → 4 → 1

NOTA

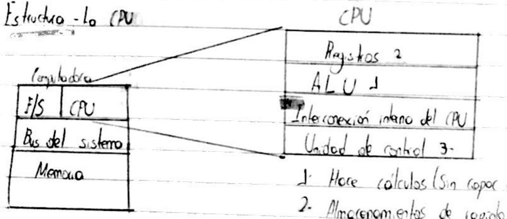
Estructura - Niv. superior


Periféricos
Computadora
Líneas de comunicax
Computadora
CPU 1
Memoria principal 2
Sistema de interconex.
3- Entrada / Salida

1- Manipulax / transformax de datos y [illegible]
2- Almacenamiento de instrucciones y datos
3- Mueve datos desde y hacia el exterior

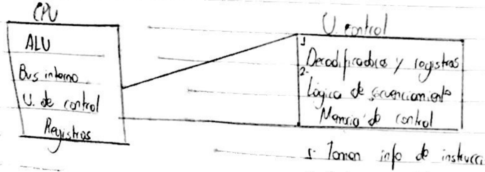
Unidos por un sistema de interconexión Proporciona [illegible] CPU, MP

Estructura - La CPU


Computadora
E/S
CPU
Bus del sistema
Memoria
CPU
Registros 2
ALU 1
4- Interconexión interna del CPU
Unidad de control 3-

1- Hace cálculos (Sin capac de decisx)
2- Almacenamientos de rapido acceso
3- Toma decisiones p/ ejecutar programas
4- Une los elem del CPU

Estructura - Unidad de control


CPU
ALU
Bus interno
U. de control
Registros
U. control
1 Decodificadores y registros
2- Lógica de secuenciamiento
Memoria de control

1- Toman info de instrucciones las órdenes q' se deben hacer
2- Instrucciones del programador se envian por la lógica de secuenc. p/ fx de la PC.

1° generax ENIAC = electronic numerical integrator and computer.

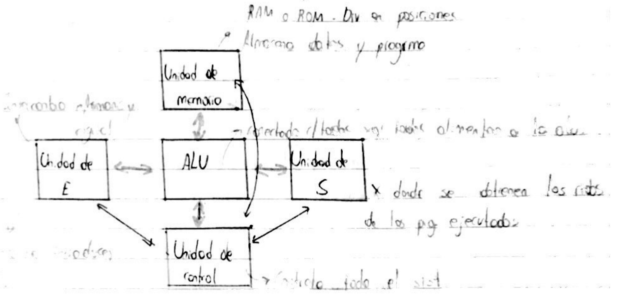
Surge como necesidad en la guerra p/ elaborar tablas de trayectoria p/ proyectiles

De 1943 y se finalizó en 1946

- Decimal
- 20 registros de 10 digitos
- Se programaba manual' mediante llaves
- 17408 tubos al vacio → Válvulas ON/OFF como llaves
- 5000 sumas/s, 360 multip/s

→ Modelo conceptual q' muestra cómo funcionan las comp.

Modelo de Von Neumann diseñó un estándar p/ las PC.

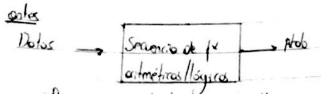

RAM o ROM. Div en posiciones
Almacena datos y programa
Unidad de memoria
Intercambio alternado y [illegible]
conectado c/todas xq' todas alimentan a la ALU
Unidad de E
ALU
Unidad de S
x donde se obtienen los rtdos de los pg ejecutados
Unidad de control
Controla todo el sist
[illegible] D c I.

Aspectos característicos de la máquina Von Neumann:

- Uso sist. binario.
  - Simplifica la implementax de fx
  - Disminuye % de fallos
- Instrucciones y D residen en memoria
  - Ejecux del programa secuencial'
  - ↑ velocidad
- Memoria direccionable x localidad sin importar el dato almacen

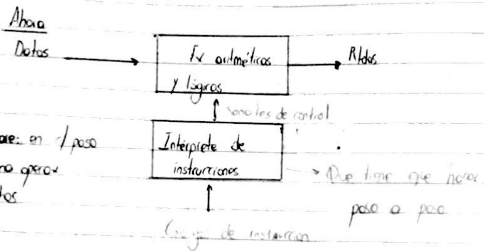
[illegible] bus de datos para todos los [illegible]

Concepto de programa antes


Datos
Secuencia de fx aritméticas/lógicas
Rtdo

- Programa en hardware: cuando ~~debemos~~ cambiar las tareas, debemos cambiar el hardware

Ahora


Datos
Fx aritméticas y lógicas
Rtdos
Señales de control
Intérprete de instrucciones
Código de instrucción
→ Que tiene que hacer paso a paso

- Programa de software: en c/ paso se efectúa alguna operac sobre los datos

- P/ c/ paso se necesita un nuevo conjunto de señales de control
- Las instrucciones proporcionan esas señales de control
- Aparece el nuevo concepto de programac → combinar instrucciones p/ <> tareas
- No hay q. cambiar hardware

¿Qué es un programa?
- Secuencia de pasos.
- Se hace operac aritmética/lógica x cada paso
- <> señales de control se necesitan p/ c/ operac.

Lo UC saca info de c/ instrucción.

IAS: Institute of Advanced study
- Memoria c/ 4096 palabras de 40 bits
- N° $C^2$
- 2 inst de 20 bits
- Set de registros
  - * " buffer de memoria (MBR)
  - * " direcciones de memoria (MAR) Dirección de memoria
  - * " Instrucción y buffer de instrucción — Contiene los 8 bits del código de operac de la instrucción q se va a ejecutar
  - * contador de programa (Program counter) PC — Dirección de la próxima pareja de instrucciones
  - * acumulador y multiplicador/cociente AC MQ — Almacenan operandos y datos temporales de la ALU

UNIVAC I: Universal Automatic computer
- 1° PC comercial
- 1° en usar un compilador p/ traducir idioma de programa a máquina
- Decimal c/ 12 dígitos x palabra
- Principal avance:
  - * sist. de cintas magnéticas q. se leían hacia atrás o adelante
  - * procedimiento de comprobac de errores
- Memoria de líneas de retardo de mercurio y tecno. a válvulas de vacío

IBM
- Equipos de procesamiento c/ tarj perforada.
- * 1953: 701 → 1° PC c/ programas almacenados de IBM
  Aplicaciones científicas
- * 1955: 702 → aplicaciones de gestión
- 1° serie de computadores 700/7000

2° generación: transistores

- Cambian los tubos al vacío
- " pequeños y baratos
- menos calor
- Dispositivos de E sólido

3° generación: circuitos integrados

- Integración a pequeña escala
  + de 100 componentes en un chip
- " a media escala
  100-3000 componentes
- " gran escala
  3000 - 100000 componentes
- " a muy gran escala
  100.000 - 100 millones de componentes

Escala s/ cant de componentes

+ 100/3000

Serie IBM 600

- Instrucciones similar
- E/S similares
- > velocidad
- Nº ↑ de E/S puertos
- > memoria
- > precio

DEC PDP-8

- 1° minicomputador
- No necesitaba aire acondicionado
- App. incrustados y OEM
- Estructura de bus

A la par de las computadoras, se fue desarrollando la fabricación de memorias

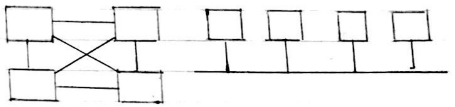
Memoria Semiconductora
- 1970
- 1° memoria c/256 bits
- Capacidad duplicada × año.
- Lectura no destructiva

Microprocesadores: Intel
- 1971 4004 - 1° microprocesador de 4 bits
  - Todo lo del CPU en un solo chip
- 1974 - 8080 1° procesador de uso genérico

Interconexión de un sist de cómputo
↓
Constituido por 3 subsistemas
- CPU
- Memoria
- E/S

- Los componentes deben comunicarse e/ sí

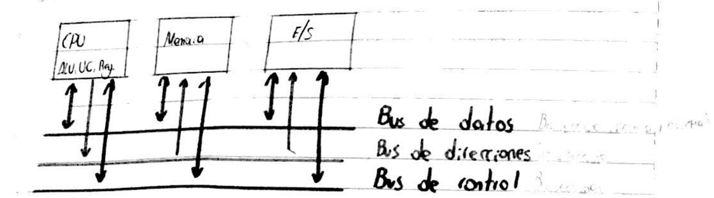
El concepto de buses tiene conexiones independientes y da rápida comunicación



- Conexiones independientes e/ <> dispositivos
- Conexión mediante un medio compartido.

Bus:
- Camino de comunicación q' conecta 1 o + dispositivos (usual: "broadcast". Requiere posicionamiento.
  Bus serie: 1 bit por vez — + ancho el bus = + info q' viaja
  Bus paralelo: conjunto en serie
- Transporta datos
- No hay diferencia e/ dato o instrucción
- El ancho es un valor determinado de las prestaciones
- 8, 16, 32, 64 bits

Interconexión a través de un bus:


CPU
ALU, UC, Reg.
Memoria
E/S
Bus de datos
Bus de direcciones
Bus de control

Direcciones
- Si el bus es compartido por <> elementos, estos deben tener identidades <>: direcciones
- La dirección de memoria identifica una celda de memoria en la q' almacena info.
- La lectura y escritura se plantean respecto de la CPU

Bus de direcciones
- Identifica el origen/destino de los datos

  La CPU necesita leer una instrucción de dada ubicac.
- El ancho de bus determina la maxima capacidad de memoria
- del sistema
- Ej. 8080 tiene bus de 16 bits dando un espacio de direcciones de 64K

El tamaño de datos celda viene dado x el bus de datos

Bus de control
- Info de control y temporizado
  - Señales de escritura / lectura de memoria o E/S
  - Señales de selección o habilitac.
  - Señales de reloj (clock)
  - Señales de pedido o interrupc.

Componentes del hardware dedicados a c/ f(x)

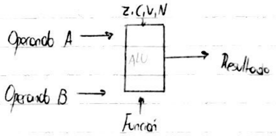
E/
- Teclado
- Mouse
- Joystick

S/
- Monitor
- Impresora

- Procesamiento
  - CPU
  - Memoria

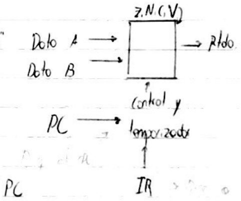
Almacenamiento
- Memoria
- Discos (rígidos, diskettes)
- Cintas, CD, DVD

CPU - ALU


Z, C, V, N
Operando A
Operando B
ALU
Resultado
Función

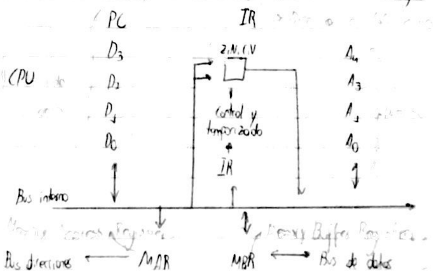
- La instrucción se almacena temporal' en un registro llamado IR
  El bloque control puede "leer" IR y así saber q' hacer, donde están los operandos y donde para rtdo.

CPU "Mejorado"


Z, N, C, V
Dato A
Dato B
Rtdo
control y temporizador
PC
IR

- Toda CPU tiene registros internos q pueden ser referenciados x el programador como fuente o destino
- "Como si" fuesen memoria pero + rápidos. Lugares de almacenamiento temporario.

CPU


PC
D3
D2
D1
D0
Z, N, C, V
control y temporizado
IR
A4
A3
A2
A0
Bus interno
Bus direcciones
MAR
MBR
Bus de datos

- La CPU interactúa con la memoria mediante registros esp.
  MAR = Reg. de direc. de memoria
  MBR = " dato de memoria
  R. conectados a los buses

- Tiene tmb otros R p/ almacenar direcc. p/ > flexibilidad

## Clase 6 Ciclo de instrucción → Surge de la ejecux de programas

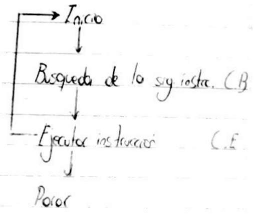
↓ Instrucciones almacenadas en la memoria q' produce D de S

El procesamiento de instrucciones se divide en 2 etapas
- Busqueda: lee desde memoria Comun a todas las instrux
- Ejecux: depende de la instrucción puede implicar varias instrucciones

La CPU procesa las instrucciones
- Debe traerlas desde una por vez
- Debe cumplir c/ operox ordenado

Ciclo de instrucción: procesamiento p/ 1 solo instrucción
- Ciclo de búsqueda:
- Ciclo de ejecux:

Ejecux interrumpido en caso de apagado o error.



## CICLO DE BÚSQUEDA y EJECUX

1. Al principio de cada ciclo, la CPU Busca una instrucción en memoria
2. En la CPU hay un registro, llamado PC, q' tiene la direx de la próxima instrucción a buscar
3. La CPU, despl de buscar c/ intrucción, incrementa el valor de PC p/ buscar la sig. instrux en secuencia
4. La instrucción buscada se carga dentro de un registro de la CPU, llamado registro de instruccion (IR)
5. La instrucción está en la forma de un codigo binario q' especifica las acciones q' tomará la CPU
6. La CPU interpreta cada instrucción y lleva a cabo las acciones requeridos

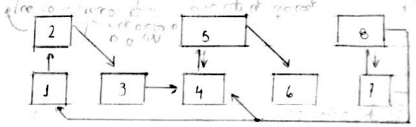
## DIFERENTES CLASES DE INSTRUCCIONES

Escritura
- CPU - MEMORIA: Datos q' pueden transferirse e/ memoria y CPU
  Lectura
- CPU - E/S: datos que pueden transferirse e/ CPU y E/S
- Procesamiento de datos: CPU efectúa operaciones aritméticas/lógicas en datos
- Control: altera la secuencia de ejecux de instrucciones

EJEMPLO
- Cargar en el registro D el contenido de la posix de memoria $940_{16}$
- Sumar el contenido de la posix de memoria $941_{16}$ al registro D y guardar rtdo en D
- Almacenar el valor del registro D en la posix $941_{16}$

Consideremos q' c/ posix de memoria almacena 16 bits. Los 1° 4 bits indican la operox a realizar, los sig. 12 bits es la direx
- 0001: 1 = cargar D desde la memoria
- 0010: 2 → almacenar D en memoria
- 0101: 5 → sumar D c/ un dato de memoria

DIAGRAMA DE ESTADOS →



- Las E en la parte superior implican un intercambio e/ CPU y la memoria
- Las E de la parte inferior implican solo op. internas en la CPU

LENGUAJE ASSEMBLY

No resulta reconfortante pensar y escribir programas como una secuencia de n°, sino que es natural verlos como una secuencia de instrucciones. P/ ello se define un lenguaje q es un mapeo de esos códigos comprensibles x un humano. A esos lenguajes se lo denomina assembly o lenguaje de ensamble. El programa q' entenderá esos códigos se llama assembler

## Clase 7 → formatos de instrucción y modos de direccionamiento.

- Elementos de una instrucción de máquina

copod 1 - Código de operox (Especifica la operox a realizar, es un código binario)

- Referencia del operando fuente, establece dónde se encuentra el operando. La operox puede involucrar 1 o + operandos fuentes (o de E)
- Referencia del operando rtdo: establece dónde almacenar el rtdo.
- Referencia de la sig. instrucción: le dice al CPU dónde buscar la sig. instrucción dsp. de la ejecux de la instrucción anterior. En la mayoría de los casos se ubica a continuox de la instrucción actual

- Los operandos fuentes y rtdo pueden estar en: memoria, registro CPU o disp de E/S.

- Representox de instrucciones:

Dentro de la computadora cada instrucción está representada mediante una secuencia de bits, que se divide en campos en correspondencia a los elementos que la componen.

Este esquema se conoce como formato de la instrucción

Es difícil p/ el programador tratar c/ representaciones binarias, por lo tanto se usa una representox simbólica

Los códigos de operox se representan x medio de abreviaturas, llamadas mnemónicos q' indican la operax.

- Los operandos tmb. pueden representarse simbólico'

Ej: mov, reg1, memoY

Instrucción que copia el valor contenido en la posix de memoria llamada memoY, a un registro denominado reg1

Código de operox , Referencia al operando , Ref. al operando

Ej x := x+y → alto nivel lenguaje

- Una instrucción de 1 niv requiere variables instrucciones de máquina. Expresa operaciones en forma concisa usando variables
- El lenguaje de máquina expresa las op. básicas usando mov. de datos y registros.
- Un lenguaje de alto niv debe convertirse a un leng. de máquina
- Un conjunto de ~~máquina~~ instrucciones de máquina debe ser capaz de expresar cualquier instrucción de 1 nivel

- Categorizox de las instrucciones
  - Procesamiento de datos → operaciones aritméticas y lógicas
  - Almacenamiento de D → transferencia dentro del sistema
  - Instrucciones E/S → transf. de D e/ PC y mecanismos externos
  - Control

Número de direcciones varía s/ la máquina

- 2 direcciones p/ los operandos. una p/ almacenar el rtdo, y de la próxima instrucción

Add , Dir Res , Dir Op1 , Dir Op 2 , Dir Próx Ins

Máquina de 4 direcciones

- Muchos campos > espacio y + larga

Máquina de 3 direcciones Add , Dir Res , DirOp1 , DirOp2 ,
- Direc. próx. inst. almacenado en CPU en el PC
- Inst largo

Máquina 2 direcciones Add , DirOp1 , DirOp2 , • Menor tamaño
- La Op1 se mueve a reg. temporal
- -dirección donde guarda rtdo.

Máquina 1 dirección Add , DirOp1
- Reg especial en CPU (acumulador)
- Instrucciones p/ carga y descarga ast. acumulador
- Un operando y rtdo en lugar predefinido
- Inst + corto

Ejemplo $a = (b+c)*d - e$

3 direcciones

```
add a, b, c
mul a, a, d
sub a, a, e
```

2 direcciones

```
mov a, b
add a, c
mul a, d
sub a, e
```

1 dirección

```
load b
add c
mul d
sub e
store a
```

Diseño del conjunto de instrucciones • Medio q' tiene el programador p/ controlar CPU

Tener en cta
- Tipos de operación (cuál y cantidad)
- Tipo de datos (cuales)
- Formato de instrucciones: longitud, nº direcciones, tamaño campo
- Registros: cant q' se pueden referenciar
- Direccionamiento: forma de especificar la dirección de un operando/instrucción

Tipos de operaciones
- Transferencia mov/load/store
- Add, sub, inc, dec, mul
- And, or, xor, not
- Conversión
- E/S. in, out
- Transferencia de control salto/bifurcación
- Control del sist usados x S.O

Tipos de datos
- Direcciones
- Nº. $\mathbb{Z}$, p fijo/flotante
- Caracteres ASCII, BCD
- Datos lógicos

Modo de direccionamiento: se usa gran cant. de bits p/ especificar origen. P/simplificar eso, se usa el modo de direccionamiento.
- Inmediato
- Directo
- x registro
- Indirecto x memoria
- Indirecto x registro
- x desplazamiento
- Del stack

s/ procesador.

1º método: si un operando va a usarse varias veces, puede colocarse en un registro. (Acceso + rápido/- bits)

2º ": especificar 1 o + operandos en forma implícita. Ej: reg2 = reg2 + fuente1, acumulador. modo disminuyen bits en la instrucción, puede no conocerse dirección hasta ejecutar programa, manejo eficiente de D.

MDD Inmediato — , operando

- El operando se obtiene automático de la memoria al mismo tiempo q' la instrucción.
- No requiere una referencia extra a memoria de datos.
- Se usa p/definir constantes y p/inicializar variable
- Tamaño de operando limitado x el tamaño del campo de direccionamiento
- Aparix de nº mágicos (nº q' no sabemos a q' obedecen)

MDD Directo. — , Dirección — Operando / Memoria

- Proporciona direx de memoria
- Espacio limitado de direx x cant. de bits del campo.
- Uso: variables globales, dirección conocida al compilar.

MDD por registro — , Direx — Operando / Registros (CPU)

- = al directo
- Uso - bits
- Pocos registros / RR preciado.

MDD indirecto por memoria — , Direc1 — Direc2 — Operando / Memoria

- c/una direc de - bits en la instrux,
- se apunta a una direc. de + bits
- > espacio de almacenamiento
- Multiples accesos a memoria

MDD indirecto por registro

, Registro — Dirección / Registro CPU — Operando / Memoria

- - bits p/ un registro q' p/ posix de memoria

Se llama apuntador

MDD x desplazamiento — , R , A — Dirección / Reg. CPU — Operando / Memoria

- Combina directo e indirecto

3 modos de direccionamiento.

- Relativo: registro referenciado = PC, la dirección de la inst. se suma al campo de direc. p/ producir direc. efectiva. El campo de direx se trata como nº Ca2
- Registro base: el registro referenciado contiene una direccion de memoria y el campo de direx tiene un desplazamiento.

Indexado: · se direcciona la memoria c/un registro + un desplazamiento.

~= q el anterior pero se intercambian papeles del registro y desplazamiento.

- Proporciona un mecanismo eficiente p/realizar op iterativos.
- Se usa un registro llamado índice (se incrementa o decrementa s/ instrux)

MDD del stack

- Arreglo lineal de localidades de memoria. Lista donde el último en entrar es el último en salir. Zona de memoria reservada.

Asociado c/ la pila o stack. hay un registro apuntador,

Pilas: gestionan las llamadas y retornos de procedimientos, y pueden contemplarse como una forma alternativa de direccionar memoria. C/0 direcciones. Conjunto de posiciones last-in-first-out.

Nº de elem. varía. Siempre se opera c/ la cabecera de la pila

El procesador se determina x las instrucciones q' ejecuta, llamada instrucciones de máquina.

Al conjunto de instrucciones <> q' puede ejecutar el procesador se llama repertorio de instrucciones del procesador

Puntero: direx tope de la pila. El puntero decrementa o incrementa s/ si se añade o elimina un elemento

Base: contiene la direx base del bloque reservado p/la pila. Si se intenta un pop c/la pila vacía, ocurre un error.

Limite de pila: direx del otro extremo del bloque reservado. Si se intenta un PUSH c/el bloque lleno, da error.

## Clase 8: Organización de registros e instrucciones

Organización de registros
- Visibles al usuario: usados x el programador. Permiten minimizar referencias a memoria principal [illegible]
- R. de control y estado: usados p/ la UC p/ controlar la op de la CPU (no son visibles p/el programador).

*El registro [illegible] al acceso a memoria y [illegible] el tiempo

No hay separax definida.

R. visibles al usuario <> tipos
- Propósito gral: el usuario pone lo que quiere → puede contener operandos.
- Datos: solo se pueden colocar datos
- Dirección: " direcciones [illegible]
- Códigos de condix: " c.d.c. (banderas)
  - ↳ Bits fijados por el hardware del procesador [illegible]

↳ Son referenciados x lenguaje de máquina

*Puede haber restricciones en los registros p/el usuario. Ej. pto flotante/fijo etc.

*Se pueden usar p/direccionamientos Ej. [illegible]

*Los registros de direx pueden ser asignados p/un modo

*Si todos los registros se usan p/ propósito gral, puede afectar al tamaño de las instrucciones

Nº de registros
- > nº reg. + bits p/ especificar en la instrucción
- Afecta al tamaño de la instrucción
- pocos reg = + referencias a memoria
- E/ 8 y 32 registros (viejo). + no hay gran mejora, > tamaño de inst.

Longitud de los registros
- De direx: ser capaz de almacenar Astt. direcc + grande
- De datos: habilitados p/ almacenar la mayoría de tipos de D
- Algunas máquinas permiten 2 reg. contiguos utilizados como un solo reg p/ almacenar vol. de doble longitud

Bits de condix
- Bits establecidos como rdo de operaciones. Gral' no alterados x el programador, no de forma directo.

---

3) puede haber R [illegible] p/ E/S de la ALU e intercambian datos c/ MBR y R. visibles

R de control y estado (no visibles) - Empleados p/ controlar la op de la CPU

Hay 4 esenciales
- PC contador de programa [illegible]
- IR registro de instrucciones. Contiene inst [illegible]
- MAR reg. de D de memoria. direx de [illegible] c/ bus de instrucciones
- MBR reg. de buffer de memoria c/ bus de datos
  - ↳ Contiene palabra de D a escribir en memoria o palabra [illegible]

Reg. usados p/ transferir datos e/ proc y memoria — Interfaz e/ procesador y buses

Estos 4 reg. se emplean p/ mov datos e/ el CPU y memoria.

Organizax de registros CPU PII Intel

| 16 | | 8 | 8 | Extended |
|---|---|---|---|---|
| CS | | AH A | x AL | EAx |
| SS | Segmentos | BH B | x BL | EBx |
| DS | Divide en áreas | CH C | x CL | ECx |
| FS | el código | DH D | x DL | EDx |
| GS | | De uso gral | | |

1) PC siempre se [illegible] el caplax de [illegible] de manera q' siempre apunta a la sig instrx a ejecutar

2) IR se carga la instrux, analiza el código de op y operando. Se intercambian datos c/ memoria mediante MAR y MBR. MAR conecta c/ el bus de direcciones. MBR bus de datos

32 bits — EIP — PC y banderas

EFLAGS

- Ax: acumulador, es el principal en op aritméticas.
- Bx: puntero base (dir base)
- Cx: contador, interviene en instrucciones de ciclo.
- Dx: datos, participa en multiplicax y división.

} Reg de propósito gral.

No significa q' [illegible] una única [illegible] de esta manera es más [illegible].

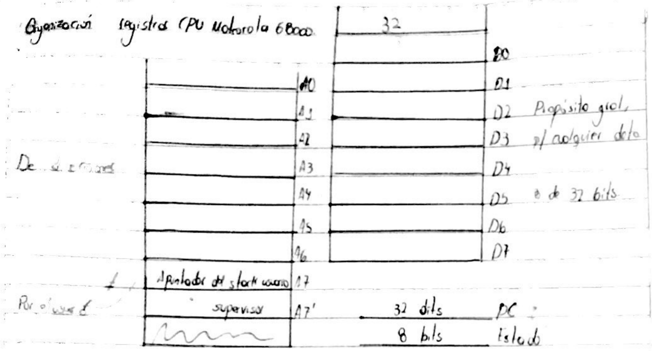
→ source index

→ destination index

Reg adicionales
- SI y DI: apuntadores q' usan las instx q' recorren arreglos o tablas. [illegible]
- BP y SP: puntero base / puntero de pila. Apuntan a zonas determinadas.
- E. extensión de R de 32 bits.

NOTA

Organización registros CPU Motorola 68000



- 8 reg de 32 bits de datos
- 9 reg de direcciones
  - 2 stacks. 1 p/ usuario. 1 p/ S.O

[illegible] de [illegible] en bits de una instr.

Instrucciones - Intel
- Forma: instrucción destino, fuente. Destino y fuente son 2 operandos, donde c/u de ellos está especificado x alguno de los mod vistos, el otro operando es un reg de la CPU
- Llamando: men: especifica de una dirección de memoria
  reg: reg de la CPU
  inm: dato inmediato

1 operando se referencia usando un modo de direccionamiento

- Instrucción men, reg. · " reg, reg · " men, inm
- Inst. reg, men · " reg, inm

- El nombre destino y fuente proviene del hecho q' si hay un mov. de datos, es desde la dcha.(fuente) hacia la izquierda (destino)

En una suma hay 2 operandos y el rdo. se almacena en el lugar del operando izquierdo (Destino)

---

no tiene direccionamiento a memoria

Instrucciones - Intel 8086
- ADD AX, BX → AX = AX+BX · MOV AL, CH → AL = CH
- ADD AL, AH → AL = AL+AH · SUB AX, BX → AX = AX-BX

- Direccionamiento por registro

reg 16 bits · 16 bits · representa en hexadecimal

% = binario
nada = decimal

- ADD AX, 35AFh → AX = AX+35AFh · MOV AL, 3Eh → AL = 3Eh 8 bits
- ADD AL, 15 → AL = AL+15 · SUB AX, 1234h → AX = AX - 1234h

- Direccionamiento inmediato

contenido de la celda

- ADD AX, [35AFh] → AX = AX+ contenido direcc 35AFh y 35B0h
- ADD AL, DATO → AL = AL + contenido variable DATO (8 bits)
- MOV CH, NUM1 → CH = contenido variable NUM1 (8 bits)

- Direccionamiento directo.

- Direc + baja = - significativa

16 bits

- ADD AX, [BX] → AX = AX+ dato almacenado en dirección contenido en BX y la q' sigue → p/ completar 16 bits.
- MOV [BX], AL → dato en la dirección contenida en BX = AL

- Direccionamiento por registro (indirecto)

2 direc → 1 [illegible]

- MOV CX, [BX+SI] → CX = dato almacenado en la direc BX+SI y la sig.
- Escritura · MOV [BX+DI], AL → Dato almacenado en BX+DI = AL

- Direccionamiento base + índice

ocupa 2 celdas

- MOV AL, [BX+2] → AL = dato almacenado en dir BX+2 lectura
- MOV [BX+2Ah], AX → dato almacenado en dir BX+2Ah y la q' sigue = AX (16 bits)

- Direccionamiento relativo por registro

- MOV AL, [BX+SI+2] → AL = dato almacenado en la dir BX+SI+2
- MOV [BX+SI+2Ah], AX = dato al " y la q' sigue = AX (16 bits)

- Direccionamiento relativo base + índice

+- largo inst simple.

Formatos de instrucción - criterios de diseño
- Instrucciones cortas o largas dependiendo bits
- N° bits/seg → ancho de banda de memoria de acceso
- Velocidad procesador/velocidad memoria
- Instrucciones + cortas "parece + rápido" el procesador
- Suficientes bits p/ expresar todos los op deseados
- La exp demuestra dejar bits libres p/ el futuro.
- Cant. de bits de datos
- La memoria puede convertirse en un cuello de botella si el procesador puede ejecutar inst + rápido q' c q toda en espera. P/ solucionarlo se usa las memorias caché

Ejemplo p/ MSX88

- Editar prueba.asm - Usar editor de textos.
- Ensamblar prueba.asm - Usar Asm88 (prueba.o A prueba.ist)
- Enlazar prueba.o - Usar link88 → Prueba.eje
- Usar MSX88 → cargar prueba.eje y ejecutar

```
ORG 2000H
MOV BX,3000H
MOV AX,[BX]
ADD BX,[02H
MOV CX,[BX]
ADD AX,CX
PUSH AX
POP DX
HLT → detener
org 3000h
db 55h, 33h, 44h, 25h
END
```

| Dir | código máquina | línea | cód. leng ensamble |
|---|---|---|---|
| 2000 | BB 00 30 | 1 | org 2000H |
| 2003 | 8B 07 | 2 | MOV BX, 300H |
| 2005 | 81 C3 02 00 | 3 | MOV AX, [BX] |
| 2009 | 8B 0F | 4 | ADD AX, CX |
| 200B | 03 C1 | 5 | PUSH AX |
| 200D | 50 | 6 | POP DX |
| 200E | 5A | 7 | HLT |
| 200F | F4 | 8 | |
| | | 9 | |
| | | 10 | |
| | | 11 | org 3000H |
| 3000 55 33 44 22 | | 12 | db 55h, 33h, 44h, 22h |
| | | 13 | end |

Símbolos

nombre: tipo: valor.

## Clase 9 → subsistemas de memoria / org mem. ppal.

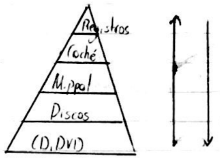
Memoria. vel. procesador: duplica c/18 meses las inst. ejecutadas / seg.

memoria: cuadruplica tamaño c/ 36 meses.

Esto genera desequilibrio el procesador y memoria. P/ equiparar esta brecha, se usan <> tipos de memorias desde rápido a caro (registros) y lento a barato (discos).

Su interac se aprovecha p/ un comportamiento equivalente al de una memoria, única, rápida y grande. (la cuál sería considerada ideal).

○ Jerarquía de memorias: forma en q' se organizan los <> tipos de memoria.



↑ + caro, + pequeña, + rápida y + frecuencia de acceso

↓ Lenta pero barata.

- P/ esta brecha tmb. se usa el caché

| Tipo | Tiempo | Tamaño |
|---|---|---|
| Registros | 1n | 1KB |
| Caché | 5-20ns | 1MB |
| Mem. ppal | 60-80ns | 1GB |

Memoria del computador:
- Tecno <>
- Fundamentos físicos <>
- Localizac <>

} Se reúnen p/ la capac. de almacenamiento tiempo de acceso reducido.

Características

Durac de la info:
- Volátil. RAM
- No volátil discos, cintas.
- Permanentes: ROM, EPROM

Modo de direccionamiento.
- Acceso x palabra: mem ppal.
- " " x bloque: discos, caché

Velocidad
- Memoria semicond.
  - Tiempo acceso tiempo el inicio de op hasta obtenido D
  - ciclo: tiemp. mínimo el instrucciones
  - $t_{ciclo} > t_{acceso}$
- Memoria magnética
  - acceso: posic cabezal + latencia
  - V. transferencia bytes/seg

Métodos de acceso
- Aleatorio. tiempo p/ acceder a una locac dada es independiente de la secuencia de accesos anteriores y es constante. Ej: mem. ppal.
- Secuencial. acceso en secuencia lineal específica.
- Directo. los bloques y registros tienen una direc única q' se basa en la locac física.
- Asociativa. memoria caché

Memoria de acceso aleatorio

RAM (Random Access Memory). Aleatorio significa q' se puede acceder a cualquier celda de memoria en el mismo tiempo, independiente de la posic en la estructura de memoria.

- SRAM memoria estática basada en flip flops.
- DRAM. dinámica basada en transistores (capacitores) Celda q' almacena D como carga eléctrica

ROM. mismo tipo de acceso.

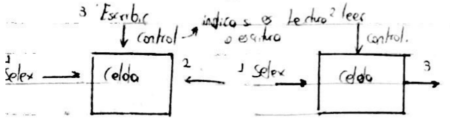
DRAM: almacena + info q' SRAM en la misma superficie. Pequeña, barata, lenta. hay q' refrescarla. p/ memoria ppal. Se refresca x tendencia natural a descargarse.

SRAM. + rápido, usada como caché

Organizac el elem básico de una memoria de semiconductor es la celda de memoria.

Todas las celdas de memoria de semiconductor comparten 3 propiedades:
- Biestable: representa 1 y 0.
- Se puede escribir en ellas, al menos 1 vez
- Se pueden leer p/ conocer el E.

c/ celda tiene 3 terminales p/ llevar una señal eléctrica.
- Selec: selecciona una celda de memoria
- Control:
- E/Lectura de D



Escritura de D

Lectura de D

Organizac de la memoria.
- Una memoria de 1 bit la implementamos c/ flip flops y "armamos" reg. sencillos de n bits c/ ff.
- P/ construir memorias "+ grandes" se requiere una org. <>, en la cual sea posible direccionar palabras individuales

Organizac del chip
- C/ chip contiene un arreglo de celdas de memoria
- En las memorias de semiconductor se han empleado 2 enfoques 2D y 2 1/2D

Organizac 2D
- El arreglo está organizado en $2^w$ palabras de B bits c/u. C/ líneas horizontal (una de $2^w$) se conecta a c/ posic de memoria, selec un renglón.
- las líneas verticales conectan c/ bit a la S.
- El decodificador q' está en el chip. tiene $2^w$ salidas p/ w E. (bits del bus de direcciones).

Organizac 2 1/2D
- El arreglo es "cuadrado" y funciona = q' 2D
- Los bits de una misma palabra están dispersos en <> chips.
- La direc se divide en 2 partes. 1 selec de renglón y una selec de columnas. Hay 2 decodificadores.

Comparac
- En 2D todos los bits están en el mismo chip.
- En 2 1/2D los bits de una misma palabra estarán en <> chips.
- 2D es largo. Nº grande de palabras de pocos bits. C/ línea de selec de palabra tiene q' tener un manejador y conectarse al decodificador. Ocupan mucha superficie
- 2D dificulta el uso eficaz de los circuitos correctores de error. El 2 1/2D al estar los bits dispersos en <> chips hay < probabilidad de error

- En 2 1/2 D las filas y columnas reducen la complejidad del los decodificadores

Problemas 1) C/ modulo cubre el espacio de direccionamiento requerido, pero solo cubre una parte de la palabra.
Solu: varios módulos en paralelo.

2) La longitud de la palabra es la deseada, pero los módulos no tienen la capac. deseada
Solu: cubrir un rango de direx c/ módulos de memoria "en serie"
C/ módulo estará en direx <>

Nuevas tecno. RAM

- La DRAM es la misma desde los 1° chips de RAM
- Enhanced DRAM [ Contiene pequeñas SRAM
  La SRAM guarda la último línea leída. (como una cache).
- Cache DRAM [ Contiene una SRAM + grande
  Se usa la SRAM como caché o como buffer serial
- synchronous DRAM (SDRAM) p/ compensar la lentitud de DRAM
  Actualm en DIMMS. Acceso sincronizado c/ reloj externo. Se presenta una direx a la RAM. RAM encuentra D (y CPU esperaría la DRAM).
  SDRAM mueve D en tiempo del reloj.
  CPU puede hacer otra cosa mientras espera.
  Burst permite al SDRAM trabajar en bloques.

Cap. 4: memoria interna.

4 = derecha
(4 → bajo
L → alto

## Práctica 5 - Lenguaje Assembly

Procesador · Interpreta 1 y 0
· Resuelve op elementales como suma, resta, ...
· Capac. acotada p/ el tamaño de D

Usuario · Difícil recordar el código binario p/ c/ instrux
Requiere resolver op. complejas.
Usa D de tamaño variable y grande.

**Lenguaje assembly** · Propone sintaxis cercana al leng "máquina" Mediante mnemónicos.
· Implementa E de control → op. lógicas + saltos condicionales s/ FLAGS.
Trata D de ≠ tamaño a → Modos de direccionamiento
una palabra c/ <> técnica
Instrucciones · cadenas binarias q' dicen q' hacer (c/ arq tiene sus inst.)
c/ inst <> representac binaria (muchas inst = códigos + largos)
c/ modo de direccionamiento → + modos = códigos de inst + largos.
tmb. es una instr <>

**Sx88** → p/ la practica *4 registros de 16 bits: AX, BX, CX, DX
A su vez pueden dividirse en 8 bits: AH, AL, BH, BL, etc.
· Tamaño de inst. variable (multiplos de 8 bits)
" de palabra de 16 bits
· Bus de D de 8 bits de ancho (2 ciclos p/ recuperar una palabra)
Direx de 16 bits ($2^{16}$ = 64 m. direcciones + 1 byte = 64 MB de memoria)
· Modos de direccionamiento Inmediato - Directo (registro) - Directo (memoria) - Ind. x reg

**Tener en cta:** · Linea ENTER al final del archivo si o si
· Nombre de archivo: máximo 8 caracteres..

**Estructura de un programa** · 1° instrucciones buscadas en direx predeterminada (2000 h)
· Los D. se ubican a partir de la dire. 1000h

ORG indica a partir de q direc. se ubican las sig. inst. y declarax

- Los D e inst. pueden estar en cualquier posix, siempre q' ubiquemos en 2000h la 1° inst. del programa

**Flags** · 8 banderas en total
- Solo afectado x op. aritmeticas y op. lógicas de la ALU
- El resto de op no afectan a los flags
- CMP altera los flags tras restar los operandos pero no almacena el rldo de ninguno de ellos.
- Serán importantes el Z, S, O, C.

signo

**Transferencia de control** Sig. inst. alteran el flujo normal del programa modificando el reg. IP s/ <> condiciones.

| Instrux | Condix | Opuesto |
|---|---|---|
| JZ | F Z= true | JNZ |
| JS | F Sig= true | JNS |
| JO | F overflow: true | JNO |
| JC | Flag carry: true | JNC |
| JMP | Incondicional | No aplica |

CPU Sx88, tiene su propio ensamblador AsM88.
P/ arrancar el programa, tecleo MSx88 <CR>

**Instrucciones**
- Inst. transf de D: MOV
- aritmético-lógicas: Inst. aritméticas ADD, ADC, SUB, SBB
  - Lógicas AND, OR, XOR, NEG, NOT
- comparax. CMP
- incremento / decremento. INC, DEC.
- cambio de flujo de programa Salto condicional: JZ, JNZ, JS, JNS, JC, JNC, JO, JNO
  - Salto incondicional JMP
  - llamadas a subrutinas CALL, RET
- " manejo de pila PUSH, POP, PUSHF, POPF.
- " gestión de las interrupciones: INT, IRET, STI, CLI
- " de control NOP, HLT
- E/S: IN, OUT

W: indica tamaño de operandos W = 0 → 8 bits DB → 1 byte / 8 bits
W: 1 → 16 bits. DW → word → 16 bits.

IP - direx de la próxima inst. a ejecutar
SP - direx del tope de la pila.

Instrucciones de salto

| | | | | | |
|---|---|---|---|---|---|
| JZ | salta si el flag | Z=1 | JNZ | salta si | Z=0; → cero |
| JS | " | S=1 | JNS | " | =0; → signo |
| JC | " | C=1 | JNC | " | C=0; → carry |
| JO | " | O=1 | JNO | " | O=0; → overflow |

JMP "direccion" ; salta siempre

Selex if then else

```
IF AL = 4 then   | CMP AL, 4      then   MOV BL, 1
 begin           | JZ then               INC CL
   BL=1;         | JMP Else              JMP Fin_IF
   CL=CL+1;      |               Else:   MOV BL, 2
 end.            |                       DEC CL
                                 Fin_IF: HLT
```